KU LEUVEN

Data Science for Business

Python and Assignment 1

D0I74A 2026–2027

Andreas Goethals



Programming in Python



Python is a programming language.
We use it to give the computer instructions.

- Mathematical operations
- Work with tables and create figures
- Build models from data

Python basics: variables

A variable stores a value so you can use it later.

```
name = "Andreas"          # text
customers = 80            # whole number
sale_rate = 0.25          # decimal
is_ready = True           # true / false

print(name)             # Andreas
```

Text needs quotes. Comments start with #. Names are case-sensitive.

Python basics: calculations

Mathematical operations with stored values, then display the result.

```
customers = 80
sales = 20
rate = sales / customers
print(rate)                 # 0.25
```

Python basics: functions

A function groups instructions so you can use them again.

```
def mean(numbers):
    return sum(numbers) / len(numbers)

numbers = [20, 30, 40]
print(mean(numbers))   # 30.0
```

Input	Function call	Output
`[20, 30, 40]`	`mean(numbers)`	`30.0`

Python basics: classes and objects

A class describes objects that hold data and have methods.

```
class DataSet:
    def __init__(self, values):
        self.values = values

    def mean(self):
        return mean(self.values)

example_data = DataSet([5, 10, 15])
print(example_data.mean())  # 10.0
```

Data

The values stored in the object

Method

A function attached to the object

Libraries for data science

A library is a collection of code you can reuse.

NumPy	Calculations with arrays
pandas	Working with tables
scikit-learn	Predictive models
Matplotlib and Seaborn	Creating figures
PyTorch and TensorFlow	Neural networks

```
import pandas as pd
```

pd is a short name for pandas.

NumPy

Calculations with many numbers at once

```
import numpy as np
values = np.array([20, 30, 40])
print(values.mean())  # 30.0
print(values + 5)     # [25 35 45]
```

An array stores the values. An operation can act on every value.

Pandas



A DataFrame is a table with rows and columns.

- Inspect and summarise data
- Select rows and columns
- Clean and prepare data

Sales data: first four rows, three columns

row	age	region	sale
0	26	North	N
1	30	North	N
2	30	North	N
3	47	South	N

```
df[["age", "region",
    "sale"]].head(4)
```

Scikit-learn



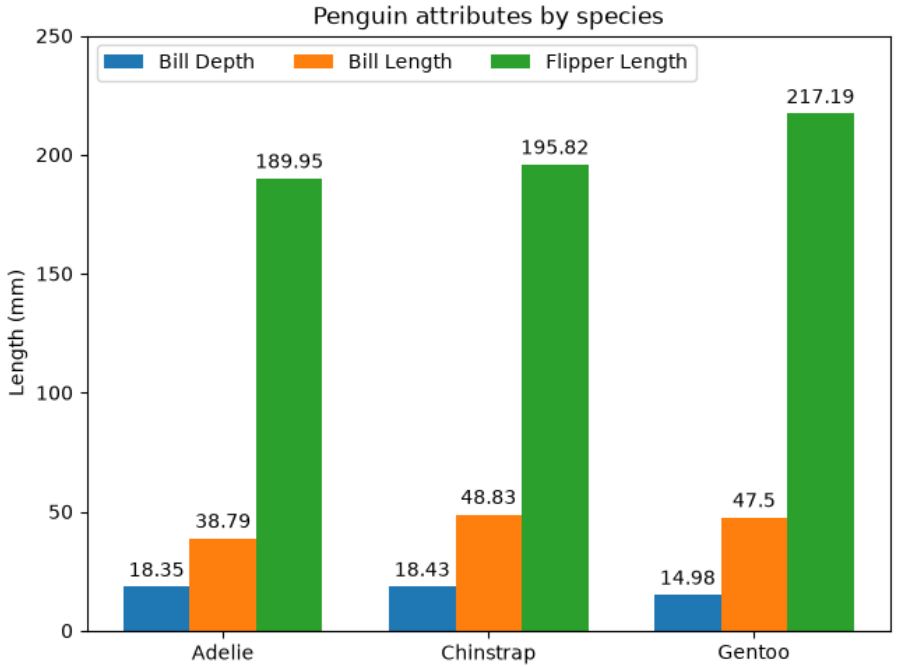
Tools to train and evaluate predictive models

Create a model	Logistic regression, decision trees and more
Fit the model	Learn from the training data
Make predictions	Apply it to new observations

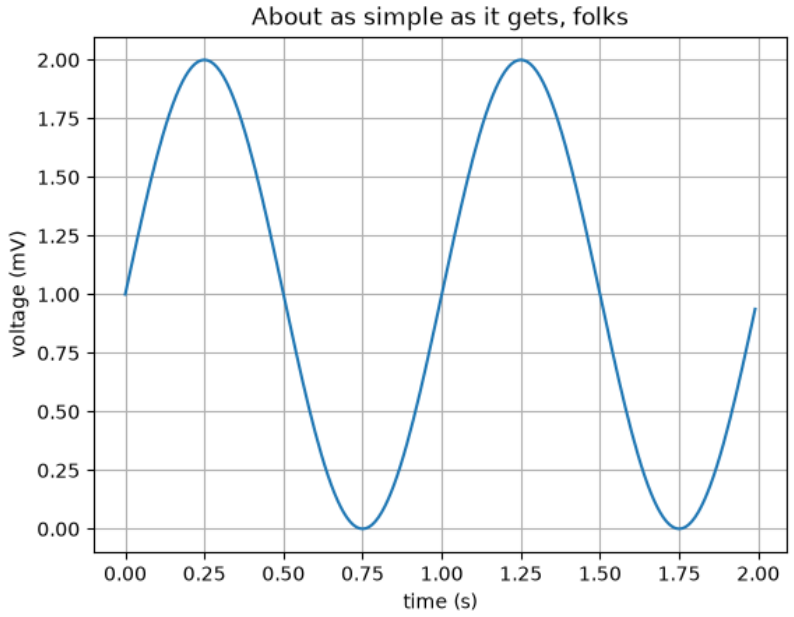
```
model = LogisticRegression()
model.fit(X_train, y_train)
probabilities = model.predict_proba(X_test)
```

Matplotlib

Create figures and control their labels, colours and layout.



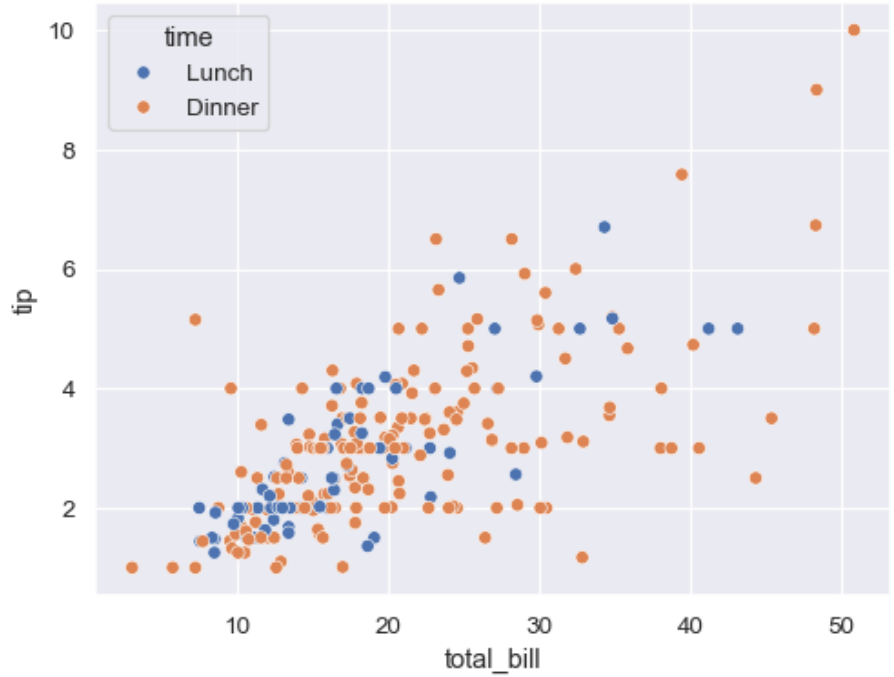
Bar charts compare groups



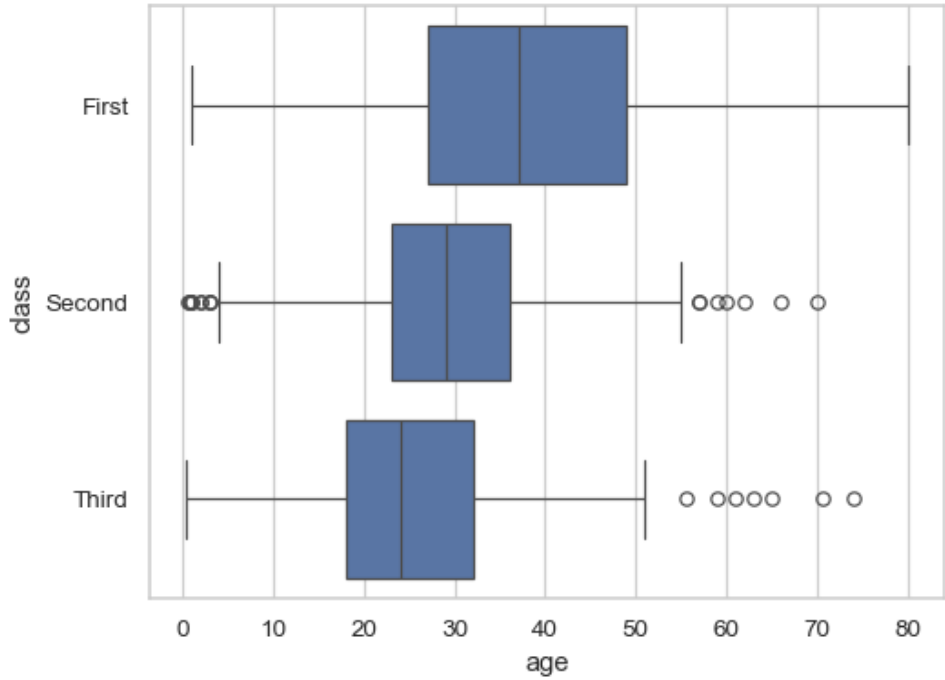
Line plots show a pattern

Seaborn

Explore relationships and distributions directly from a table.



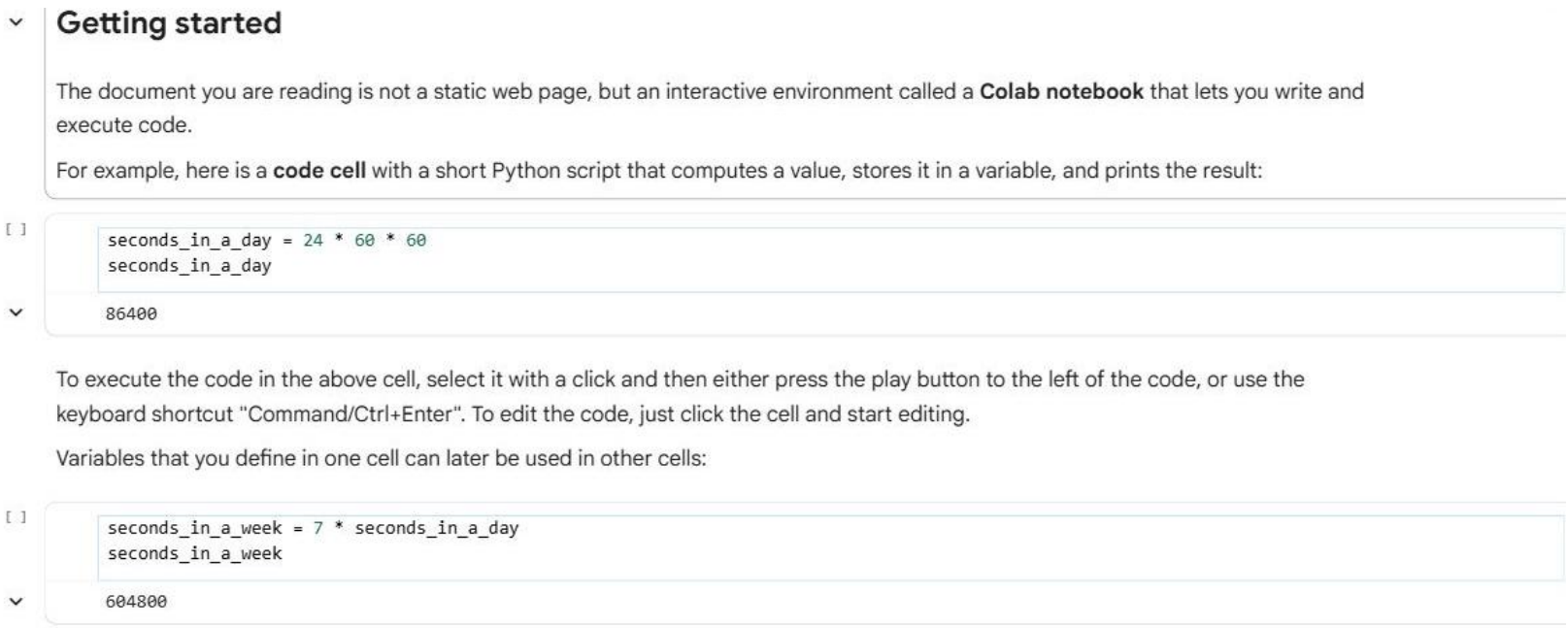
Relationships between variables



Distributions across groups

Python notebooks

One .ipynb file combines text, code and results.



Getting started

The document you are reading is not a static web page, but an interactive environment called a **Colab notebook** that lets you write and execute code.

For example, here is a **code cell** with a short Python script that computes a value, stores it in a variable, and prints the result:

```
[ ]
seconds_in_a_day = 24 * 60 * 60
seconds_in_a_day
```

```
86400
```

To execute the code in the above cell, select it with a click and then either press the play button to the left of the code, or use the keyboard shortcut "Command/Ctrl+Enter". To edit the code, just click the cell and start editing.

Variables that you define in one cell can later be used in other cells:

```
[ ]
seconds_in_a_week = 7 * seconds_in_a_day
seconds_in_a_week
```

```
604800
```

Text cells

Markdown format & adds headings and formatting.

Code cells

Run Python.
See the output below.

Google Colab

Python notebooks in your browser

No local Python installation needed

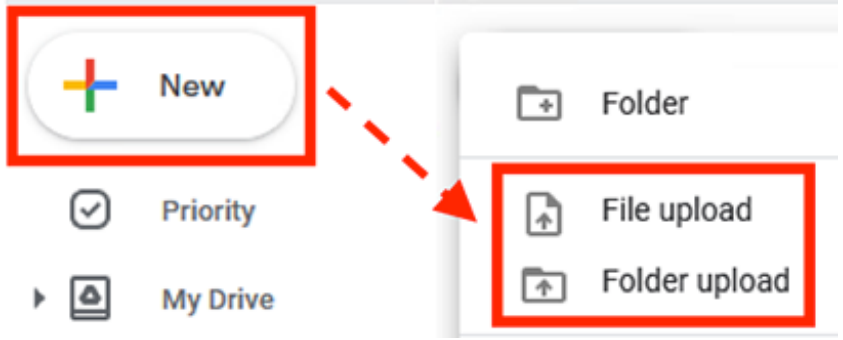
Save and share notebooks through Google Drive



Colab: getting started and connecting

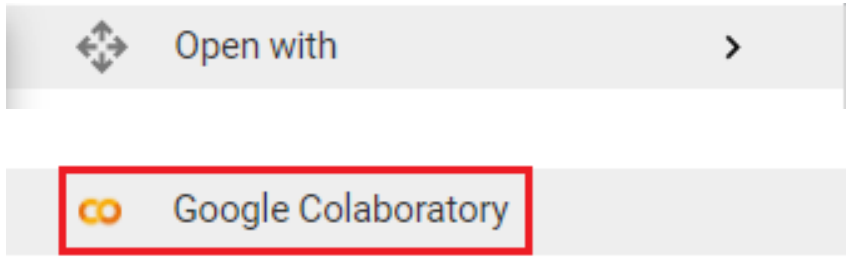
1. Upload the notebook to Drive

Sign in at drive.google.com.
Choose New > File upload and select the .ipynb file from your computer.



2. Open the notebook in Colab

Right-click the file in Drive. Choose Open with > Google Colaboratory.
The notebook opens in Colab.

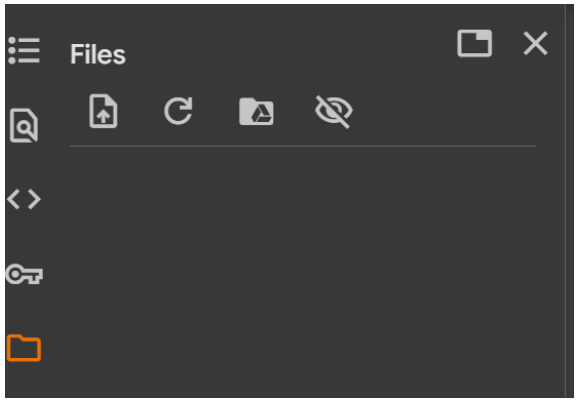


3. Connect to a runtime

Click Connect at the top right.
Wait until connected, then run a code cell.



Colab: uploading and reading data



1. Upload the dataset

Open Files and click the upload icon.
Select the CSV and wait for its filename to appear in the panel.

2. Read the file into a table

```
import pandas as pd
DATA_PATH = "Sales Demonstration Dataset.csv"
df = pd.read_csv(DATA_PATH)
df.head()
```

Python looks in the working folder. Elsewhere? Use the file menu’s Copy path.

LLMs for programming

ChatGPT, Gemini and other LLMs can help write and explain code.

A useful prompt names the data and the result you want.

“Plot sale_rate by region from region_summary.
Show percentages and label each bar with its sample size.”

Ask for explanations and check the output against your data.

Gemini inside the notebook

Click the Gemini spark icon in Colab to open the chat panel.



Live demo: Sales data and Gemini

- Upload the Sales CSV and inspect its contents
- Compare sale rates and group sizes with pandas
- Ask Gemini for a chart, then check and explain it

Assignment 1: guidelines

Teams of two

Deadline

10 November 2026

23:59 Belgian time

Complete the supplied notebook in Python.
Use the reporting template and export it to PDF.

One email per team to andreas.goethals@kuleuven.be

Assignment 1: your four files

File	What you do with it
A1_Assignment.pdf	Read the questions and guidelines
A1_Notebook.ipynb	Complete this notebook in Python
A1_Template.docx	Fill in the report and export to PDF
South_German_Credit.csv	Load the dataset in the notebook

Assignment 1: credit risk

South German Credit

1,000 observations and 21 columns, including the credit-risk outcome

1	**Explore**	Choose two variables. Compare bad-credit rates and group sizes.
2	**Model**	Select variables and use WOE. Build and evaluate logistic regression.
3	**Score**	Build the scorecard. Show the test scores in boxplots.

Explain your choices and what your figures show.

What a good submission shows

- You understand the results and explain your decisions
- Your report uses the provided three-page template
- Your figures are clear and your notebook runs from top to bottom
- You check any code or interpretation suggested by an LLM

KU LEUVEN

Questions ?

